

Roteiro de Apresentação N1 — MY Service Parking

Disciplina: Projeto e Arquitetura de Software | **Período:** 2026/2 **Apresentador:** Marcos Vinícius (Código na IDE + Live Demo no Navegador) **Parceiro:** Yan (Apresentação de Slides e Diagramas C4) **Tempo Total da Apresentação**

Prática: ~10 minutos

Cronograma Geral (10 Minutos)

Bloco	Tempo	Foco Principal	Onde Mostrar
1. Abertura & Setup	00:00 – 01:00 (1 min)	Conexão com os C4 do Yan + Estrutura do Projeto e Terminais	VS Code / Estrutura de Pastas
2. Código: Gateway & Microserviços	01:00 – 03:00 (2 min)	Roteamento Proxy, Guard JWT e Comunicação entre Serviços	VS Code (Back-end)
3. Código: Front-end SPA	03:00 – 04:30 (1.5 min)	Interceptors do Axios, Router Guards e Separação de Regras	VS Code (Front-end)
4. Live Demo: Fluxo Operacional	04:30 – 08:30 (4 min)	Login, Vagas, Mensalistas, Entrada por Placa e Saída com Cobrança	Navegador (localhost:5173)
5. Relatórios & Fechamento	08:30 – 10:00 (1.5 min)	Consolidação de dados, RNF cumpridos e Encerramento	Navegador + VS Code

Checklist Pré-Apresentação (Faça 5 minutos antes)

- ☐ **7 Terminais Rodando** (usar o script do PowerShell do README ou terminais individuais):

- Porta 3000: API Gateway
- Porta 3001: auth-service
- Porta 3002: estacionamento-service
- Porta 3003: mensalista-service
- Porta 3004: pagamento-service
- Porta 3005: vaga-service
- Porta 5173: Front-end Vue 3 (<http://localhost:5173>)

/*

cd frontend npm run dev

*/

/*

```
ROOT = (Get - Location).Path
servicos = @( @{"dir"="services\auth-service";
label="auth-service"}, @{"dir"="services\estacionamento-service";
label="estacionamento-service"}, @{"dir"="services\mensalista-service";
label="mensalista-service"}, @{"dir"="services\pagamento-service"; label="pagamento-
service"}, @{"dir"="services\vaga-service"; label="vaga-service"}, @{"dir"="gateway";
label="gateway"} )
foreach ($s in servicos) { Start-Process powershell -ArgumentList "-
NoExit", "-Command", "cd 'ROOT$(s.dir)'; npm run dev" -WindowStyle Normal }
Start-Sleep 4
Start-Process powershell -ArgumentList "-NoExit", "-Command", "cd
'ROOT\frontend'; npm run dev" -WindowStyle Normal
Write-Host "Todos os servicos iniciados! Acesse: http://localhost:5173"
```

*/

- ☐ **Arquivos-chave abertos em abas no VS Code** (para não perder tempo procurando):

1. [gateway/src/routes/index.js](#)
2. [gateway/src/middleware/auth.js](#)

3. `services/estacionamento-`

`service/src/controllers/estacionamentoController.js`

4. `services/pagamento-`

`service/src/controllers/pagamentoController.js`

5. `frontend/src/services/api.js`

6. `frontend/src/router/index.js`

- ☐ **Navegador aberto em `http://localhost:5173/login`** (em modo limpo, com o DevTools/Console aberto se quiser mostrar as requisições de rede).



Roteiro Passo a Passo

Bloco 1: Abertura & Visão Geral da Arquitetura na IDE (00:00 – 01:00)

O que fazer na tela:

1. Comece compartilhando a tela do **VS Code**.
2. Abra a árvore de diretórios do projeto no explorer lateral (`MY_SP/`).
3. Mostre rapidamente os terminais rodando na parte inferior.

O que falar:

"Obrigado, Yan. Agora que vimos a modelagem conceitual nos diagramas C4, vou mostrar como essa arquitetura foi materializada no código e como ela opera em tempo de execução.

Aqui no VS Code, vocês podem observar a nossa divisão estrita de responsabilidades: temos a pasta `gateway/`, a pasta `services/` contendo cada um dos 5 microsserviços independentes (`auth-service`, `estacionamento-service`, `mensalista-service`, `pagamento-service` e `vaga-service`), e a pasta `frontend/` com a nossa SPA em Vue 3.

Cada serviço possui seu próprio `package.json`, seu `.env` e roda em portas isoladas (3000 a 3005), comunicando-se exclusivamente via protocolo HTTP REST."

Bloco 2: Código do Back-end — Gateway & Microserviços (01:00 – 03:00)

Arquivos para mostrar no VS Code:

1. `gateway/src/routes/index.js` e `gateway/src/middleware/auth.js`

- **Onde clicar:** Linhas de proxy e middleware de rotas protegidas.
- **O que falar:**

*"Começando pelo coração da nossa entrada de dados: o **API Gateway**. O front-end só conhece a porta 3000. Como vemos aqui em `gateway/src/routes/index.js`, as rotas de `/auth` são públicas para login, mas todas as outras rotas de negócio — entradas, saídas, mensalistas, pagamentos e vagas — passam obrigatoriamente pelo `authMiddleware`. Se abirmos `auth.js`, o Gateway valida o JWT com `jwt.verify()`. Se for inválido ou ausente, a requisição é barrada imediatamente com erro 401 antes mesmo de tocar nos microserviços."*

2. `services/estacionamento-service/src/controllers/estacionamentoController.js`

- **Onde clicar:** Função `verificarMensalista()` (~linha 34) e `registrarEntrada()` (~linha 53).
- **O que falar:**

*"Aqui temos um exemplo prático da regra arquitetural: **bancos de dados isolados**. O `estacionamento-service` não acessa tabelas do `mensalista-service` diretamente. Na função `verificarMensalista()`, ele faz uma requisição HTTP interna para a porta 3003 do mensalista. Se a placa for de um mensalista inadimplente, o acesso é negado (403). Se for rotativo ou mensalista em dia, a entrada é gerada."*

3. `services/pagamento-service/src/controllers/pagamentoController.js`

- **Onde clicar:** Função `calcularValor()` (~linha 11) e rota `/pagamentos/calcular`.
- **O que falar:**

"No *pagamento-service*, isolamos a lógica financeira: tarifa de R\$ 8,00 por hora com tolerância/carência de 10 minutos (RN-06). Nenhum cálculo de dinheiro é feito no front-end — o front apenas solicita o cálculo ao serviço de pagamento e exibe o resultado."

● Bloco 3: Código do Front-end — SPA Vue 3 (03:00 – 04:30)

🔗 Arquivos para mostrar no VS Code:

1. *frontend/src/services/api.js*

- **Onde clicar:** Request & Response Interceptors do Axios.
- **O que falar:**

"No front-end, construído em Vue 3 com Vite, centralizamos a comunicação no *api.js*. Utilizamos **interceptors do Axios**: no Request Interceptor, injetamos automaticamente o Bearer Token JWT armazenado no *LocalStorage*. No Response Interceptor, se qualquer serviço retornar 401 (token expirado), o usuário é automaticamente deslogado e redirecionado para o login."

2. *frontend/src/router/index.js*

- **Onde clicar:** *router.beforeEach* (Navigation Guard).
- **O que falar:**

"O *vue-router* implementa guards de navegação nas rotas. Se o usuário tentar acessar */dashboard* ou */vagas* sem autenticação, a rota é interceptada e protegida no lado do cliente."

● Bloco 4: Live Demo no Navegador — Fluxo Operacional Completo (04:30 – 08:30)

🔗 O que fazer na tela:

Altere para o navegador em <http://localhost:5173>.

1. Login & Autenticação (04:30 – 05:15)

- **Ação:**
 - Digite: `admin@myparking.com` / `admin123`.
 - Clique em **Entrar**.
- **O que falar:**

"Vamos iniciar a demonstração prática pelo fluxo de autenticação. Ao logar com as credenciais de administrador, o `auth-service` gera o token JWT assinado, que o Gateway valida e o Pinia armazena na sessão. Com isso, o menu lateral com todas as permissões é liberado."

2. Dashboard e Mapa de Vagas (05:15 – 06:00)

- **Ação:**
 - Mostre os cards de métricas no **Dashboard** (Veículos no Pátio, Vagas Livres, Entradas/Saídas).
 - Clique no menu **Vagas**.
 - Mostre os 2 andares e os tipos de vagas (Comum, PCD, Idoso, Elétrico).
- **O que falar:**

"No Dashboard, temos a visão executiva do pátio alimentada pelo `vaga-service` e `estacionamento-service`. Na tela de Vagas, vemos o status em tempo real das 20 vagas distribuídas em 2 andares, com identificação para vagas especiais (PCD, Idoso e Carro Elétrico), atendendo ao requisito RF11 e já preparando o sistema para os sensores IoT da Fase 2."

3. Gestão de Mensalistas (06:00 – 06:45)

- **Ação:**
 - Clique no menu **Mensalistas**.
 - Destaque o mensalista `João Silva` (Placa: `XYZ9K88` - **EM DIA**).
 - Destaque a mensalista `Maria Santos` (Placa: `DEF5G67` - **INADIMPLENTE**).
- **O que falar:**

"Aqui na tela de Mensalistas, temos o controle de contratos. Notem que temos o João Silva com status 'EM DIA' e a Maria Santos com status

'INADIMPLENTE'. Vamos ver como o sistema se comporta ao tentar dar entrada para esses veículos."

4. Registro de Entrada de Veículos (06:45 – 07:30)

- **Ação 1 (Teste de Bloqueio por Inadimplência):**

- Vá para a tela **Entrada**.
- Digite a placa da Maria Santos: DEF5G67 e clique em Confirmar.
- **Resultado:** O sistema exibe o alerta de bloqueio / inadimplência.

- **O que falar:**

"Vejam que ao tentar registrar a placa DEF5G67, o estacionamento-service consultou o mensalista-service e barrou o acesso imediatamente com erro 403, cumprindo a regra de negócio RN-03."

- **Ação 2 (Entrada com Sucesso - Rotativo):**

- Digite uma nova placa: BRA2E19.
- Clique em **Confirmar Entrada**.
- **Resultado:** Voucher gerado com data, hora, tipo ROTATIVO e código de comprovante.

- **O que falar:**

"Agora digitando uma placa rotativa BRA2E19, a entrada é confirmada instantaneamente, o voucher do cliente é emitido e o pátio é atualizado."

5. Registro de Saída, Cálculo de Tarifa e Pagamento (07:30 – 08:30)

- **Ação:**

- Vá para a tela **Saída**.
- Selecione ou digite a placa de um veículo em aberto (ex: ABC1D23 ou a placa recém-criada).
- Clique em **Localizar e Calcular Cobrança**.
- Mostre a apuração do tempo de permanência e o valor calculado.
- Selecione a forma de pagamento: **PIX** ou **Dinheiro** (informe valor recebido para demonstrar o cálculo do troco).
- Clique em **Confirmar Pagamento e Liberar Saída**.

- **O que falar:**

*"Na tela de saída, o atendente busca a placa. O sistema recupera a hora de entrada e chama o **pagamento-service**. Aqui vemos o cálculo automático da permanência com a carência de 10 minutos aplicada. Ao escolher 'Dinheiro' e informar o valor pago, o sistema calcula o troco exato. Ao confirmar, a movimentação é finalizada, a vaga é liberada no pátio e o comprovante de saída é gerado."*

Bloco 5: Relatórios & Conclusão (08:30 – 10:00)

O que fazer na tela:

1. Clique no menu **Relatórios**.
2. Mostre o resumo consolidado do dia (total de entradas, saídas, faturamento acumulado e ticket médio).
3. Volte rapidamente para a IDE ou deixe na tela de Dashboard.

O que falar:

*"Para fechar a visão operacional, na tela de **Relatórios** o gestor tem a consolidação em tempo real de todas as movimentações, taxa de ocupação e faturamento do dia.*

Em resumo, demonstramos que:

1. *A aplicação segue rigorosamente os 4 níveis do Modelo C4 apresentados pelo Yan;*
2. *Implementamos uma arquitetura de microserviços desacoplada com API Gateway centralizado;*
3. *Garantimos os Requisitos Não Funcionais prioritários: Segurança via JWT, Confiabilidade nas transações e Escalabilidade com serviços isolados;*
4. *O sistema entrega 100% das funcionalidades do MVP da Fase 1 e já está preparado arquiteturalmente para receber os sensores e cancelas IoT na Fase 2.*

Com isso, concluímos a nossa demonstração e ficamos à disposição para as perguntas do professor e dos colegas. Obrigado!"

Guia Rápido de Placas para Testes Rápidos

Placa	Tipo	Status	Finalidade do Teste na Apresentação
DEF5G67	Mensalista	Inadimplente	Demonstrar bloqueio automático na Entrada (403)
XYZ9K88	Mensalista	Em Dia	Demonstrar entrada autorizada de mensalista sem cobrança
ABC1D23	Rotativo	No Pátio	Demonstrar busca rápida e saída com cobrança de horas
BRA2E19	Rotativo	Novo	Demonstrar cadastro de nova entrada e emissão de voucher



Dicas de Ouro para a Apresentação

- Ritmo e Firmeza:** Não precisa correr; 10 minutos é tempo suficiente se você seguir as abas já pré-abertas.
- Vocabulário Técnico:** Use termos como *"API Gateway"*, *"Proxy reverso"*, *"JWT Bearer Token"*, *"Desacoplamento de Domínio"*, *"Interceptors do Axios"* e *"Idempotência"*. O avaliador valoriza muito isso em Arquitetura de Software.
- Caso dê algum imprevisto na rede/terminal:** Lembre-se que o Gateway possui tratamento de erro 502 customizado — caso algum microserviço caia, mostre o erro elegante no terminal e reinicie apenas aquele serviço específico, destacando que *o restante do sistema permaneceu de pé* (isolamento de falhas).